claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a8b8ccd86903b3ab3.jsonl`
- SHA-256: `3db7ce1464df6c46f4b0566df9103488fd57e5b9b4aca36a1a6eedc5e76900c8`
- Source modified: `2026-06-20T09:02:28+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`

Transcript

1. user (2026-06-20T09:00:58.225Z)

Adversarially verify this review finding against the ACTUAL code in the working tree. Default to 'false-positive' unless you can point to the concrete line(s) and a real failure scenario.
Context: Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master.
The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the worker.
Files changed/added (read them in the WORKING TREE ? changes are uncommitted):
- backend/pipeline/detectors/native_wasb_runner.py (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py (added --device, threaded device into _build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py (_resolve_runner: new 'native' branch + _build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py (_build_native_runner override; WASB substrate only)
- backend/config/models.py (WasbIndexConfig.device + .python_bin; detector_impl doc mentions 'native')
- backend/worker/__main__.py (cmd_train: --trajectory-source {synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)

DESIGN INTENT / CONTRACTS (judge against these):
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner, WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively (no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename '{stem}__{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) -> identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-platform path handling matters.
- weights/repo/python/scratch come from env first (WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.

FINDING [worker-extract] (major) @ backend/worker/__main__.py:252 (cmd_train) -> backend/pipeline/segmenters/trajectory_hybrid.py:88-95 (_probe_fps_width): For the detector path, fps/frame_width come from _probe_fps_width, which uses cv2.VideoCapture and SILENTLY falls back to (30.0, 1280.0) when cv2 can't open the file or reports 0 (unusual codec, container cv2 can't read but ffmpeg/WASB can, or any open failure). These values are written verbatim into the

training manifest and the gen0 harness uses them for frame->time mapping
(build_frame_arrays(pts, frame_width, fps) + features(arr, fps)).
WHY: A wrong fps silently mis-aligns EVERY rally label against the trajectory: if the real video
is 60fps but the probe falls back to 30, every frame-indexed trajectory time is doubled relative
to the golden labels ? the harness trains/evals on corrupted alignment with no error surfaced.
get_video_duration (used 3 lines earlier in the segmenter) already uses robust ffprobe; the fps
probe does not. This is training-data corruption that looks like a normal run.
PROPOSED FIX: For the detector path, fail loud on a probe miss instead of silently defaulting:
detect the fallback (cv2 returned <=0) and skip the video with a clear warning (parity with the
'no trajectory' skip), or add/usean ffprobe-based fps/width probe. Do not feed a guessed 30/1280
into a REAL trajectory's time mapping.

2. assistant (2026-06-20T09:01:02.902Z)

I'll verify this finding against the actual code. Let me start by pulling the repo and reading
the relevant files.

3. user (2026-06-20T09:01:03.646Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import os
22      import sys
23      import tempfile
24      from typing import Any, List
25
26      from backend import storage
27      from backend.config import load_config
28      from backend.infrastructure.database import Database
29      from backend.pipeline.segmenters.base import segmenter_registry
30      from backend.pipeline.validators.base import registry as validator_registry
31      from backend.reporting import emit_indexer_report
32      from backend.results import Failure
33      from backend.utils.hashing import compute_video_id
34
35
36      def _coerce(v: str) -> Any:
37          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
38          low = v.lower()
39          if low in ("true", "false"):
40              return low == "true"
41          for cast in (int, float):
```

```
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
```

```
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(video_id, local_video, "failed" if failures else "processed",
                     [r.model_dump() for r in val_results])

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print("[worker] validation FAILED: "
                      + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
                return 2
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[worker] unknown segmenter: {seg_name!r} "
                      f"(have {list(segmenter_registry.list_available())})")
                return 2
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return 3
            segments = result.value if hasattr(result, "value") else result
            status = "success"
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0

```

```
161
162     def _resolve_train_detector(args: argparse.Namespace, config: Any):
163         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
164         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
165         stub=CPU mock). Returns the runner, or prints an error + returns None.
166
167         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
168         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
169         no shuttle detector and is rejected.
170         """
171         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
172
173         seg_cls = segmenter_registry.get(args.segmenter)
174         if not seg_cls:
175             print(f"[worker] unknown segmenter: {args.segmenter!r} "
176                   f"(have {list(segmenter_registry.list_available())})")
177             return None
178         seg = seg_cls(None, config)
179         if not isinstance(seg, TrajectoryHybridSegmenter):
180             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
181                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
182             return None
183         runner, _ = seg._resolve_runner(config.get("indexing", {}))
184         ok, msg = runner.healthcheck()
185         if not ok:
186             print(f"[worker] detector environment not ready: {msg}")
187             return None
188         print(f"[worker] detector ready ({args.segmenter}): {msg}")
189         return runner
190
191
192     def cmd_train(args: argparse.Namespace) -> int:
193         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
194         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
195
196         Two trajectory sources (the train pipeline + harness are identical either way):
197           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
198             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
200             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
201             is the M3.5 "first real training" path; only the trajectory source flips.
202         """
203         import subprocess
204
205         config = load_config(args.config) if args.config else load_config()
206         _apply_overrides(config, args.set)
207         storage_cfg = config.storage.model_dump()
208
209         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
210         labels_dir = os.path.join(scratch, "labels")
211         traj_dir = os.path.join(scratch, "trajectories")
212         videos_dir = os.path.join(scratch, "videos")
213         os.makedirs(labels_dir, exist_ok=True)
214         os.makedirs(traj_dir, exist_ok=True)
215
216         corpus = args.corpus_uri.rstrip("/")
```

```
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
                from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
                fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
```

```
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
        pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
        pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
        pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument("--set", action="append", default=[], metavar="k=v",
                        help="config override, e.g. --set indexing.skip_ai_handoff=true")
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
        pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
                        "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
        pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
```

```
323         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325                              "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326                              "(needs videos/ in the corpus; honours detector_impl).")
327         pt.add_argument("--segmenter", default="wasb_hybrid",
328                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330         pt.add_argument("--config", default=None)
331         pt.add_argument("--scratch", default=None)
332         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333         pt.add_argument("--frame-width", type=float, default=1280.0,
334                         help="synth trajectory frame width (detector probes the video)")
335         pt.add_argument("--set", action="append", default=[], metavar="k=v")
336         pt.set_defaults(func=cmd_train)
337         return p
338
339
340     def main(argv=None) -> int:
341         args = build_parser().parse_args(argv)
342         return args.func(args)
343
344
345     if __name__ == "__main__":
346         sys.exit(main())
347
```

4. user (2026-06-20T09:01:03.943Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
```

```
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          seconds = max(0, int(seconds))
39          h, rem = divmod(seconds, 3600)
40          m, s = divmod(rem, 60)
41          return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44      class TrajectoryHybridSegmenter(BasePlaySegmenter):
45          """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47          #: config section under `indexing` (velocity_threshold lives there), the
48          #: output subdir for trajectories, and the metadata detector-tag prefix.
49          family: str = ""
50          #: human-readable label used in log lines.
51          display_label: str = ""
52
53          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54              """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
55              raise NotImplementedError
56
57          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
58              """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
59              native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
60              raise NotImplementedError(
61                  f"detector_impl='native' is not supported for the "
62                  f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
63                  f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65          def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
66              """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68              ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69              ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70              is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71              in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72              ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73              (the WSL runner) so the default production path is unchanged
74              (``docs/DECOUPLED_COMPUTE/``).
75              """
76              impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77              if impl == "stub":
78                  from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79                  logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                              self.display_label or "trajectory")
81                  return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
```

```
{"detector_impl": "stub"}
82              if impl in ("native", "native_wasb", "wasb_native"):
83                  logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                              self.display_label or "trajectory")
85                  return self._build_native_runner(idx_cfg)
86              return self._build_runner(idx_cfg)
87
88          @staticmethod
89          def _probe_fps_width(video_path: str) -> tuple:
90              """Return (fps, frame_width) for a video, with sensible fallbacks."""
91              cap = cv2.VideoCapture(video_path)
92              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94              cap.release()
95              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97          @staticmethod
98          def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99              """Provider-reported exchange count, else a duration-based estimate.
100
101             One canonical implementation ? the twins had drifted (wasb relied on
102             ``int(None)`` raising into the fallback; same result, by accident).
103             """
104             shot_count = segment.get("shuttle_exchange_count")
105             if shot_count is None:
106                 return max(3, int(duration / 0.8))
107             try:
108                 return int(shot_count)
109             except (ValueError, TypeError):
110                 return max(3, int(duration / 0.8))
111
112         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
113         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                     frame_width: float, video_path: str,
115                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
117             returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119             features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
120             return {
121                 "peak_velocity": cw["peak_velocity"],
122                 "mean_velocity": cw["mean_velocity"],
123                 "active_frame_count": cw["active_frame_count"],
124                 "track_id_count": cw["track_id_count"],
125             }
126
127         def _select_for_handoff(self, windows: List[Dict[str, Any]],
128                                 window_stats: List[Dict[str, Any]],
129                                 idx_cfg: Dict[str, Any]) -> List[int]:
130             """Indices of the candidate windows that proceed to the AI handoff (and thus may
131             become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132             ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
133             Persisted candidates are NOT affected ? they remain the full superset for
offline
134             re-scoring."""
135             return list(range(len(windows)))
136
137         def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
138                           player_pool: Optional[List[str]] = None,
139                           max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
140             # override-ignore: the ABC declares the Result contract (Success|Failure)
```

```
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.get("indexing", {})
            # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
            # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
            # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
            # measure it against the Gemini path, or to avoid the cloud entirely). With
            # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
            skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            if skip_ai:
                model_name = "local-noai"
                logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
                            "be emitted as rallies; no %s handoff, no API key needed.",
                            label, provider_name)
            else:
                model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    from backend.pipeline.segmenters._keyhint import api_key_hint
                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        + api_key_hint(api_key_env_var)
                    )
                    return []
                try:
                    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
            # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            runner, runner_params = self._resolve_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
```

```
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.get("max_window_duration", 0.0)
            window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
            window_hard_cap = idx_cfg.get("window_hard_cap", False)
            window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
            inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
            inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
            window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
            window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                return []
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
                max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
```

```
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)

            ai_segments: List[dict] = []
            if skip_ai:
                # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
                # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
                # falls back to the duration-based estimate (no AI exchange count).
                ai_segments = [
                    {
                        "start_time": all_candidate_windows[wi]["start"],
                        "end_time": all_candidate_windows[wi]["end"],
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
                            "(no AI handoff).", len(ai_segments))
            else:
                handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info("Phase 3: %s validation on %d candidate windows...",
                            provider_name, len(handoff_indices))

                def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                    w_start = max(0, window["start"] - handoff_padding)
```

```
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                        return []
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
                                                         label=f"Handoff {wi+1}")
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                    futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                               for wi in handoff_indices]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    metadata = {
                        "shot_count": self._shot_count_from(segment, dur),
                        "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                        "detector": f"{self.family}-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
                    if segment_id:
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append({
                            "id": segment_id, "start_time": start, "end_time": end,
                            "duration": dur, "metadata": metadata,
                        })
                except Exception as e:
                    logger.warning("Skipping invalid segment struct: %s", e)
```

```
371
372             return segments_data
373
```

5. assistant (2026-06-20T09:01:06.516Z)

Now let me look at the existing `get_video_duration` (the robust ffprobe one mentioned in the finding) and the gen0 harness to confirm how fps/frame_width flow into training.

6. user (2026-06-20T09:01:07.447Z)

```
1       import subprocess
2       import logging
3       from typing import Optional
4
5       logger = logging.getLogger(__name__)
6
7       def get_video_duration(video_path: str) -> float:
8           """Returns the exact duration of a video file in seconds using ffprobe.
9           Returns 0.0 on failure."""
10          try:
11              duration_output = subprocess.check_output([
12                  "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                  "-of", "default=noprint_wrappers=1:nokey=1", video_path
14              ]).decode().strip()
15              return float(duration_output)
16          except Exception as e:
17              logger.error(f"Could not detect video duration for {video_path}: {e}")
18              return 0.0
19
20      def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                              output_path: str, reencode: bool = False,
22                              scale_width: Optional[int] = None) -> bool:
23          """Extracts a chunk from a video file.
24
25          Args:
26              reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                        If False, uses stream-copy (fast but cuts at nearest keyframe).
28                        Use True for short clips where keyframe drift matters.
29              scale_width: If set (requires reencode), downscale to this width (height auto,
30                        even). Useful for high-res sources headed to an API that analyses at
31                        low res anyway ? shrinks uploads with no analysis loss.
32          """
33          cmd = [
34              "ffmpeg", "-y",
35              "-ss", str(start_time),
36              "-t", str(duration),
37              "-i", video_path,
38          ]
39          if reencode:
40              if scale_width:
41                  cmd += ["-vf", f"scale={int(scale_width)}:-2"]
42              cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
43          else:
44              cmd += ["-c", "copy"]
45          cmd.append(output_path)
46          try:
47              subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
48              return True
49          except Exception as e:
50              logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
51              return False
52
```

7. user (2026-06-20T09:01:08.075Z)

```
training\gen0\__init__.py
training\gen0\harness.py
```

8. assistant (2026-06-20T09:01:12.649Z)

The finding claims `get_video_duration` uses robust ffprobe (confirmed - lines 7-18) while `_probe_fps_width` uses cv2 with silent fallback (confirmed - lines 88-95). Now let me verify the harness consumes fps/frame_width for time-mapping.

9. user (2026-06-20T09:01:13.243Z)

```
1       """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3           python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4               --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6       Productionizes the rally_seq prototype per ADR-008:
7       - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8         honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9         the FINAL model on all videos.
10      - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11            <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12            <out>/<version>/manifest.json   (the reviewable contract: feature-schema version,
13                                             calibration, training lineage + GT hashes,
14                                             eval results, attestations, gate verdict)
15      - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16        (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17        (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18        undefined => no self-declared victory). The product side owns final admission
19        (backend/eval/regression.py); this harness never grades itself into serving.
20
21      Training may import backend.* (one-way wall); backend must never import training.*.
22      """
23      from __future__ import annotations
24
25      import argparse
26      import hashlib
27      import json
28      import os
29      import subprocess
30      import time
31      from math import comb
32      from typing import Dict, List, Optional
33
34      import numpy as np
35
36      from backend.eval.classifier import LogisticRegression
37      from backend.eval.gt_loader import load_gt_intervals
38      from backend.eval.regression import gt_hash
39      from backend.eval.rally_seq_proto import (
40          labels_for, run as lovo_run, _seg_f1,
41      )
42      from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
43      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
44
45      # Post-processing constants ? part of the model's identity, recorded in the manifest.
46      CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
47      THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]
48
49
```

```
    def _git_sha() -> str:
        try:
            return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
                                  text=True, timeout=10).stdout.strip() or "unknown"
        except Exception:
            return "unknown"


    def _sha256(path: str) -> str:
        h = hashlib.sha256()
        with open(path, "rb") as f:
            for chunk in iter(lambda: f.read(65536), b""):
                h.update(chunk)
        return h.hexdigest()


    def sign_test(learned: List[float], floor: List[float]):
        """Paired one-sided sign test: is learned > floor more often than chance?

        At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
        wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
        pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv]  # ties drop,
standard
        wins = sum(1 for lv, fv in pairs if lv > fv)
        n = len(pairs)
        p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
        return wins, n, p


    def train_final(specs: List[Dict]):
        """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).

        The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
        vids = []
        for s in specs:
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
            X, times = features(arr, float(s["fps"]))
            gts = load_gt_intervals(s["labels"], strict=False)
            vids.append({"name": s["name"], "X": X, "times": times,
                         "y": labels_for(times, gts), "gts": gts})
        Xall = np.vstack([v["X"] for v in vids])
        yall = np.concatenate([v["y"] for v in vids])
        clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
        clf.fit(Xall, yall, feature_names=FEAT_NAMES)
        thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
        return clf, float(thr)


    def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
        """Pre-flight ship-gate.

        Provisional F1 target (NOT owner-ratified; calibration tracked in #172).
        Floor (necessary): heuristic LOVO ~0.480 (from quality table).
        Multi-court ref (makes owned \"worth serving\" per data): 0.66.
        Proposed for \"very high P/R\" + significance: F1@tIoU0.5 >= 0.70 on
        multi-court folds with paired-sign p<0.05 vs heuristic (refine as corpus grows).
        Beating floor + sign test necessary; C3 main commercial blocker.
        ship=True only when all non-C3 blocks clear (honesty first).
        """
        reasons: List[str] = []
        floor_passed = None
        sign = None
```

```
        if floor:
            floor_mean = float(floor["mean_f1"])
            floor_passed = eval_res["mean_f1"] > floor_mean
            if not floor_passed:
                reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
            per = floor.get("per_video", {})
            paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
            if paired:
                wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
                sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
                if p >= 0.05:
                    reasons.append(f"paired sign test not significant ({wins}/{n} wins,
p={p:.3f}) "
                                   f"? grow the corpus before claiming superiority")
        else:
            reasons.append("no floor baseline provided ? nothing to beat")

        # Provisional blocker (per owner review on #170; see #172 for calibration).
        # (The number itself is not the blocker; calibration tracked in #172.)
        reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
        reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
        # ship remains False until C3 resolved + other gates (by design).
        return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}


    def run(manifest_path: str, out_dir: str, version: str,
            floor_path: Optional[str] = None) -> dict:
        with open(manifest_path, encoding="utf-8") as f:
            specs = json.load(f)
        floor = None
        if floor_path:
            with open(floor_path, encoding="utf-8") as f:
                floor = json.load(f)

        # 1) Honest LOVO eval (the number that matters).
        eval_res = lovo_run(specs)

        # 2) Final model on all videos.
        clf, thr = train_final(specs)

        # 3) Gate (pre-flight).
        verdict = gate_verdict(eval_res, floor)

        # 4) Artifact bundle.
        bundle_dir = os.path.join(out_dir, version)
        os.makedirs(bundle_dir, exist_ok=True)
        weights_path = os.path.join(bundle_dir, "weights.json")
        payload = clf.to_dict()
        payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
        payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
        with open(weights_path, "w", encoding="utf-8") as f:
            json.dump(payload, f, indent=2)

        manifest = {
            "artifact": {
                "name": "rally_tal_gen0", "version": version,
                "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
                "git_sha": _git_sha(),
                "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
            },
```

```
171             "feature_schema": FEATURE_SCHEMA,
172             "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
173                             "note": "threshold tuned train-side on the full corpus; "
174                                     "honest generalization = the LOVO eval below"},
175             "training_lineage": {
176                 "corpus_manifest": manifest_path,
177                 "split_protocol": "leave-one-video-out by match",
178                 "label_source": "human (rally-annotator golden CSVs)",
179                 "videos": [{"name": s["name"], "labels": s["labels"],
180                             "gt_hash": gt_hash(load_gt_intervals(s["labels"],
strict=False)),
181                             "fps": s["fps"]} for s in specs],
182             },
183             "eval": {
184                 "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
185                 "per_video": eval_res["per_video"],
186                 "mean_f1": eval_res["mean_f1"],
187                 "floor": floor,
188             },
189             "attestation": {
190                 "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
191                 "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
192                 "no_paid_llm_training_labels": True,
193                 "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
194             },
195             "gate": verdict,
196         }
197         manifest_out = os.path.join(bundle_dir, "manifest.json")
198         with open(manifest_out, "w", encoding="utf-8") as f:
199             json.dump(manifest, f, indent=2)
200 
201         print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
202         print(f"weights : {weights_path}")
203         print(f"manifest: {manifest_out}")
204         print(f"GATE    : ship={verdict['ship']}  floor_passed={verdict['floor_passed']}  "
205               f"sign_test={verdict['sign_test']}")
206         for b in verdict["blockers"]:
207             print(f"  - {b}")
208         return manifest
209 
210 
211     def main() -> None:
212         ap = argparse.ArgumentParser(description=__doc__)
213         ap.add_argument("--manifest", required=True,
214                         help="corpus manifest JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
215         ap.add_argument("--floor", default=None,
216                         help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
217         ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
218         ap.add_argument("--version", required=True, help="semver for the artifact bundle")
219         args = ap.parse_args()
220         run(args.manifest, args.out, args.version, floor_path=args.floor)
221 
222 
223     if __name__ == "__main__":
224         main()
225 
```

10. assistant (2026-06-20T09:01:16.917Z)

This confirms the harness uses fps/frame_width for frame->time mapping. Let me verify the chain: `lovo_run(specs)` (the actual eval number) and `build_frame_arrays` / `features` consumption.

Let me read `rally_seq_proto.py` and `rally_seq.py` features.

11. user (2026-06-20T09:01:17.847Z)

```
1       """PROTOTYPE: a learned per-FRAME rally segmenter (vs the unsupervised windowing
heuristic).
2
3       Motivation (2026-06-08 finding): WASB tracks the shuttle well on Kushagra (97% of GT
4       rallies contain >=2 net-crossings) yet NO velocity-windowing config segments it
5       (per-video F1 ceiling 0.163), while it works on Adarsh (0.73). The rally SIGNAL is in
6       the trajectory; the hand-tuned heuristic just can't extract it across footage. This
7       prototype tests the hypothesis that a LEARNED model on per-frame trajectory features
8       recovers the rally segments ? including on the footage the heuristic fails on.
9
10      Approach (deliberately small, dependency-light: numpy + scipy + the repo's pure-python
11      LogisticRegression ? no torch/sklearn):
12        1. Per-frame features over a local window from the WASB trajectory: visibility rate,
13           mean/max shuttle speed (frame-width-normalized), net-crossing RATE, x/y spread.
14           All are scale/camera-robust RATES (unlike a raw velocity threshold).
15        2. Per-frame label = inside a human GT rally or not.
16        3. LEAVE-ONE-VIDEO-OUT: train the classifier on the other match(es), test on the
17           held-out one (the honest cross-footage number; no within-video leakage).
18        4. Smooth probabilities -> threshold (tuned on TRAIN) -> contiguous in-rally segments
19           -> merge tiny gaps / drop short -> score vs GT at IoU>=0.5 (same metric as the
heuristic).
20
21      This is a directional PROOF-OF-SIGNAL, not a production model (n=2 videos, linear model,
22      hand features). It exists to answer one question: is the rally boundary learnable from
23      the trajectory where the heuristic fails? Compare its held-out F1 to the heuristic LOVO.
24
25      Run:
26          python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
27      """
28      from __future__ import annotations
29
30      import json
31      import argparse
32      from typing import Dict, List, Tuple
33
34      import numpy as np
35      from scipy.ndimage import uniform_filter1d
36      from scipy.stats import rankdata
37
38      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
39      from backend.eval.classifier import LogisticRegression
40      from backend.eval.calibrate_wasb import load_golden_gts
41      from backend.eval.metrics import score
42      # Feature extraction is single-sourced in backend/features (ADR-008): training and
43      # serving import the SAME code, so train == serve by construction. Names re-exported
44      # here for back-compat with existing imports/tests.
45      from backend.features.rally_seq import (  # noqa: F401 (re-exports)
46          FEAT_NAMES, FEATURE_SCHEMA, FEATURE_SCHEMA_VERSION, _spread,
47          build_frame_arrays, features,
48      )
49
50      Interval = Tuple[float, float]
51
52
53      def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
54          y = np.zeros(len(times))
55          for i, t in enumerate(times):
56              for s, e in gts:
57                  if s <= t <= e:
58                      y[i] = 1.0
59                      break
```

```
        return y


    def roc_auc(y: np.ndarray, p: np.ndarray) -> float:
        """Rank-based AUC (Mann-Whitney) with MIDRANK tie handling.

        Uses scipy.stats.rankdata (average method) so tied probabilities get averaged ranks
?
        without this, an all-tied classifier scored a spurious order-dependent 0.25..0.75
        instead of 0.5, and dead-time frames produce identical all-zero feature rows
(guaranteed
        ties), so the headline AUC carried an unquantified bias."""
        pos, neg = p[y > 0.5], p[y <= 0.5]
        if pos.size == 0 or neg.size == 0:
            return float("nan")
        ranks = rankdata(p)  # midranks (ties averaged)
        return float((ranks[y > 0.5].sum() - pos.size * (pos.size + 1) / 2) / (pos.size *
neg.size))


    def probs_to_segments(probs: np.ndarray, times: np.ndarray, threshold: float,
                          merge_gap: float = 1.0, min_dur: float = 1.0, smooth: int = 7) ->
List[Interval]:
        p = uniform_filter1d(probs, max(1, smooth), mode="nearest")
        on = p >= threshold
        runs: List[Interval] = []
        i, n = 0, len(on)
        while i < n:
            if on[i]:
                j = i
                while j + 1 < n and on[j + 1]:
                    j += 1
                runs.append((float(times[i]), float(times[j])))
                i = j + 1
            else:
                i += 1
        merged: List[Interval] = []
        for s, e in runs:
            if merged and s - merged[-1][1] <= merge_gap:
                merged[-1] = (merged[-1][0], e)
            else:
                merged.append((s, e))
        return [(s, e) for s, e in merged if e - s >= min_dur]


    def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
        f1s = []
        for v in vids:
            segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
            f1s.append(score(segs, v["gts"], 0.5).f1)
        return sum(f1s) / max(len(f1s), 1)


    def run(specs: List[Dict]) -> dict:
        data = {}
        for s in specs:
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
            X, times = features(arr, float(s["fps"]))
            gts = load_golden_gts(s["labels"])
            data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
gts}
        names = list(data)
        print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
videos) ===")
```

```
119         print(f"features: {FEAT_NAMES}\n")
120
121         results = []
122         for test in names:
123             train = [data[n] for n in names if n != test]
124             Xtr = np.vstack([v["X"] for v in train])
125             ytr = np.concatenate([v["y"] for v in train])
126             clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
127             clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
128             # tune the decision threshold on TRAIN segment-F1 (no test leakage)
129             thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
float(t)))
130             v = data[test]
131             probs = clf.predict_proba(v["X"])
132             auc = roc_auc(v["y"], probs)
133             segs = probs_to_segments(probs, v["times"], float(thr))
134             sc = score(segs, v["gts"], 0.5)
135             # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
ceiling
136             # from the threshold-transfer gap. Optimistic ? for diagnosis only.
137             othr = max(np.arange(0.2, 0.81, 0.05),
138                        key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
139             osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
140             results.append({"video": test, "thr": float(thr), "auc": auc,
141                             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
142                             "n_pred": len(segs), "n_gt": len(v["gts"]),
143                             "oracle_thr": float(othr), "oracle_f1": osc.f1})
144             print(f"[held-out {test}]  per-frame AUC {auc:.3f}  |  LOVO-threshold F1
{sc.f1:.3f} "
145                   f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train)  |  "
146                   f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
147         mean_f1 = sum(r["f1"] for r in results) / len(results)
148         print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
149         return {"per_video": results, "mean_f1": mean_f1}
150
151
152     def main() -> None:
153         ap = argparse.ArgumentParser(description=__doc__)
154         ap.add_argument("--manifest", required=True, help="JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
155         args = ap.parse_args()
156         with open(args.manifest, encoding="utf-8") as f:
157             run(json.load(f))
158
159
160     if __name__ == "__main__":
161         main()
162
```

12. user (2026-06-20T09:01:18.062Z)

```
1       """Per-frame rally features over a shuttle/ball trajectory ? the rally_seq feature
schema.
2
3       THE single source of truth for the Gen-0 TAL feature extraction (ADR-008): the training
4       harness (training/) and the serving path import THIS module, so train == serve by
5       construction. The fps-normalization bug fixed on 2026-06-10 (speed read ~2x smaller at
6       59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.
7
8       Pure numpy/scipy ? importable without torch/cv2. Any change to the feature semantics
9       MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
10      trained with and the serving loader refuses a mismatch.
11      """
```

```
    from __future__ import annotations

    from typing import Dict

    import numpy as np
    from scipy.ndimage import uniform_filter1d, maximum_filter1d

    # Bump on ANY semantic change to build_frame_arrays/features (names, normalization,
    # windowing, occlusion rule). Artifacts pin this; the loader hard-fails on mismatch.
    FEATURE_SCHEMA_VERSION = 1

    FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]

    # Default windowing ? part of the schema (manifests record the values actually used).
    DEFAULT_WIN_SEC = 0.75
    DEFAULT_STRIDE_SEC = 0.1

    FEATURE_SCHEMA = {
        "id": "rally_seq",
        "version": FEATURE_SCHEMA_VERSION,
        "feat_names": FEAT_NAMES,
        "win_sec": DEFAULT_WIN_SEC,
        "stride_sec": DEFAULT_STRIDE_SEC,
        # Explicit because its absence was a real bug: speed is fraction-of-frame-width
        # PER SECOND (* fps), matching the heuristic windowing brain.
        "fps_semantics": "per-second",
        "occlusion_rule": "speed/crossings not computed across gaps > max(2,
round(0.25*fps)) frames",
        "input": "trajectory points (frame, visible, x, y) + REQUIRED runtime metadata {fps,
frame_width}",
    }


    def _spread(a: np.ndarray) -> float:
        return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0


    def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
        """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.

        Speed is normalized to fraction-of-frame-width PER SECOND (``* fps``), matching the
        heuristic windowing brain ? previously it was per-frame, so the same physical
shuttle
        speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
transfer
        on a corpus that mixes frame rates. Speed/crossings are also not computed across a
long
        occlusion gap (mirrors the heuristic resetting on invisibility)."""
        pts = sorted(points, key=lambda p: p.frame)
        N = (pts[-1].frame + 1) if pts else 1
        vis = np.zeros(N)
        x = np.zeros(N)
        y = np.zeros(N)
        for p in pts:
            if 0 <= p.frame < N and p.visible and p.x >= 0:
                vis[p.frame] = 1.0
                x[p.frame] = float(p.x)
                y[p.frame] = float(p.y)

        mask = vis > 0
        play_is_x = _spread(x[mask]) >= _spread(y[mask])
        coord = x if play_is_x else y
        net = float(np.median(coord[mask])) if mask.any() else 0.0
        deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)

```

```
72          speed = np.zeros(N)
73          spd_mask = np.zeros(N)
74          cross = np.zeros(N)
75          max_gap = max(2, round(0.25 * fps))  # frames; a longer gap = occlusion break, not
motion
76          prev = None
77          for p in pts:
78              f = p.frame
79              if not (0 <= f < N and vis[f]):
80                  continue
81              if prev is not None:
82                  df = f - prev
83                  if 0 < df <= max_gap:
84                      speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
85                      spd_mask[f] = 1.0
86                      c, pc = coord[f] - net, coord[prev] - net
87                      if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
88                          cross[f] = 1.0
89              prev = f
90          return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)
91
92
93      def features(arr: Dict, fps: float, win_sec: float = DEFAULT_WIN_SEC,
94                   stride_sec: float = DEFAULT_STRIDE_SEC):
95          """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
96          N = arr["N"]
97          fw = arr["fw"]
98          size = max(3, int(win_sec * fps) * 2 + 1)
99          vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
100
101         n = uniform_filter1d(vis, size, mode="constant")                        # mean
visibility (== vis_rate)
102         sm = uniform_filter1d(spd_mask, size, mode="constant")
103         spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
104         spd_max = maximum_filter1d(speed, size, mode="constant")
105         cross_rate = uniform_filter1d(cross, size, mode="constant") * fps       # crossings
/ second
106         xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
107         x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
108         x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
109         ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
110         y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
111         y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
112
113         idx = np.arange(0, N, max(1, int(stride_sec * fps)))
114         X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
115         return np.nan_to_num(X), idx / fps
116
```

13. assistant (2026-06-20T09:01:29.135Z)

This is conclusive. The harness's frame->time mapping critically depends on fps:
- `build_frame_arrays(pts, frame_width, fps)` ? line 84: speed normalization `/ frame_width / df * fps`, line 75: `max_gap = max(2, round(0.25 * fps))`
- `features(arr, fps)` ? line 98: `size = max(3, int(win_sec * fps) * 2 + 1)`, line 105: `cross_rate * fps`, line 113-115: `idx = arange(0, N, int(stride_sec * fps))` and `times = idx / fps`

The `times` array (line 115: `idx / fps`) maps trajectory frame indices to seconds, and `labels_for(times, gts)` (proto line 53-60) compares those times against the golden label

intervals. A wrong fps directly mis-aligns every label.

Now let me confirm the exact call site and the cv2 fallback behavior. The finding pinpoints `__main__.py:252` and `_probe_fps_width` at lines 88-95. Both confirmed. Let me verify the stub/synth path uses explicit fps (so this is specific to the detector path) and check whether `_probe_fps_width` is wrapped in any error handling at the call site.

14. user (2026-06-20T09:01:30.264Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3       `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4       segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5       on the droplet against MinIO.
6       """
7       import json
8
9       import pytest
10
11      from backend.config import load_config
12      from backend.worker import __main__ as wmain
13
14
15      # ---------------------------------------------------------------- pure helpers
16
17      @pytest.mark.parametrize("raw,expected", [
18          ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
"wasb_hybrid"),
19      ])
20      def test_coerce(raw, expected):
21          assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)
22
23
24      def test_apply_overrides_on_typed_config():
25          cfg = _fresh_config()
26          wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
27                                       "sport=tennis"])
28          assert cfg.indexing.skip_ai_handoff is True
29          assert cfg.indexing.min_segment_duration == 1.5
30          assert cfg.sport == "tennis"
31
32
33      def test_apply_overrides_rejects_bad_format():
34          cfg = _fresh_config()
35          with pytest.raises(ValueError):
36              wmain._apply_overrides(cfg, ["no_equals_sign"])
37
38
39      def test_write_intervals_csv(tmp_path):
40          segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
41                  {"start_time": 9.0, "end_time": 12.5}]
42          out = tmp_path / "intervals.csv"
43          wmain._write_intervals_csv(segs, str(out))
44          lines = out.read_text().splitlines()
45          assert lines[0] == "start,end,shot_count,ending_reason"
46          assert lines[1].startswith("2.000,5.000,4,")
47          assert lines[2].startswith("9.000,12.500,,")
48
49
50      def test_write_report_json(tmp_path):
51          out = tmp_path / "report.json"
52          wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
```

```
str(out))
53          data = json.loads(out.read_text())
54          assert data["video_id"] == "vid1" and data["segment_count"] == 1
55          assert data["segments"][0] == {"start": 1.0, "end": 2.0}
56
57
58      def test_parser_infer_accepts_set_and_uris():
59          args = wmain.build_parser().parse_args([
60              "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
61              "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
62          assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
63          assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
64
65
66      # ------------------------------------------------------------- cmd_infer (local,
mocked)
67
68      class _OKResult:
69          passed = True
70          validator_name = "fake"
71          message = "ok"
72
73          def model_dump(self):
74              return {"validator_name": "fake", "passed": True, "message": "ok"}
75
76
77      class _FailResult(_OKResult):
78          passed = False
79          message = "too blurry"
80
81
82      class _FakeSeg:
83          def __init__(self, db, config):
84              pass
85
86          def process_video(self, video_id, path, max_frames=None):
87              return [
88                  {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
89                  {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},
90              ]
91
92
93      def _fresh_config():
94          # Load defaults without touching any cwd config.json.
95          import tempfile
96          p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
97          p.write("{}")
98          p.close()
99          return load_config(p.name)
100
101
102     @pytest.fixture
103     def mocked_pipeline(monkeypatch):
104         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
105         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
106                             lambda path, cfg: ([_OKResult()], {}))
107         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
108         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
109
110
111     def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
112         video = tmp_path / "in.mp4"
113         video.write_bytes(b"FAKEVIDEO")
```

```
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        out = tmp_path / "bucket"
        scratch = tmp_path / "scratch"

        rc = wmain.main([
            "infer", "--video-uri", str(video), "--out-uri", str(out),
            "--config", str(cfg), "--scratch", str(scratch),
            "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
        ])
        assert rc == 0
        intervals = out / "outputs" / "vid123" / "intervals.csv"
        report = out / "outputs" / "vid123" / "report.json"
        assert intervals.exists() and report.exists()
        rows = intervals.read_text().splitlines()
        assert len(rows) == 3                          # header + 2 rallies
        assert json.loads(report.read_text())["segment_count"] == 2


    def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                            lambda path, cfg: ([_FailResult()], {}))
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"X")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path /
"o"),
                         "--config", str(cfg), "--scratch", str(tmp_path / "s")])
        assert rc == 2


    # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---

    def test_cmd_train_orchestration(tmp_path, monkeypatch):
        import os
        import subprocess

        labels_dir = tmp_path / "corpus" / "labels"
        labels_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        (labels_dir / "v1.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
        (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)
        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
```

```
str(out_dir),
                         "--version", "0.0.1", "--config", str(cfg), "--scratch",
str(scratch)])
        assert rc == 0
        # bundle pushed to <out-uri>/weights/<version>/
        assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
        assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
        # the manifest fed to the harness enumerated both labeled videos with a trajectory
each
        specs = json.loads((scratch / "manifest.json").read_text())
        assert len(specs) == 2
        assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)


    def test_parser_train_detector_args():
        args = wmain.build_parser().parse_args([
            "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version",
"1.0.0",
            "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
        assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"


    def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
        """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
With
        RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
runner)."""
        import os
        import subprocess

        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no
GPU/WSL)
        corpus = tmp_path / "corpus"
        labels_dir = corpus / "labels"
        videos_dir = corpus / "videos"
        labels_dir.mkdir(parents=True)
        videos_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
        (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
        (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
        (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)
        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                         "--version", "0.0.2", "--config", str(cfg), "--scratch",
str(scratch),
                         "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
        assert rc == 0
        specs = json.loads((scratch / "manifest.json").read_text())
        assert len(specs) == 2
```

```
233         # Real-extract path: each spec points at an on-disk trajectory the runner actually
wrote.
234         assert all(os.path.exists(s["trajectory"]) for s in specs)
235         assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub
runner's output
236
```

15. assistant (2026-06-20T09:01:36.442Z)

The test for the detector source uses the stub runner with fake video bytes (`b"FAKEVIDEO1"`). Note: the stub path still calls `_probe_fps_width` on a fake non-video file ? which would trip exactly the fallback the finding describes (cv2 can't open `b"FAKEVIDEO1"`, so fps=0 -> falls back to 30.0). The test passes only because the stub doesn't care about fps correctness. This confirms the silent-fallback path is real and is even hit by the test, just not asserted on.

Let me confirm there's no ffprobe-based fps probe elsewhere in the codebase that the finding's proposed fix could leverage, and double-check the synth path uses explicit `args.fps` (not the probe) so the finding correctly scopes this to the detector path only.

16. user (2026-06-20T09:01:37.303Z)

```
README.md:320:This usually means validation didn't run (it's the step that probes the file via
`ffprobe`), or `ffprobe` couldn't read the stream's `r_frame_rate`. To fix: confirm the real fps
?
README.md:322:ffprobe -v error -select_streams v:0 -show_entries stream=r_frame_rate -of csv=p=0
"C:/path/to/video.mp4"
backend\worker\__main__.py:252:            fps, frame_width =
TrajectoryHybridSegmenter._probe_fps_width(local_video)
backend\main.py:425:    fps = cap.get(cv2.CAP_PROP_FPS)
tests\test_fusion_hybrid.py:48:         patch.object(FusionSegmenter, "_probe_fps_width",
return_value=(30.0, 1280.0)), \
docs\GOLDEN_REGRESSION_FIXTURES.md:132:- Green proves only that the **deterministic post-
trajectory transforms** are unchanged for the **frozen cases**. It does **not** cover: the
WASB/TrackNet detector, `_probe_fps_width`/cv2 frame-seek, the AI/provider handoff, chunking/re-
encode, the ffmpeg stitch, DB persistence/migrations, or the API filter. *A refactor that breaks
decoding or the detector passes these gates green.* The owner-box `regression.py` DB run remains
the only full-chain check.
docs\CODE_MAP.md:89:(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner;
`_probe_fps_width` fallback (30.0, 1280.0).)
docs\CODE_MAP.md:290:- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` /
runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes
`59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
docs\CODE_MAP.md:350:| `test_tracknet_hybrid.py` |
`pipeline/segmenters/tracknet_hybrid.TrackNetHybridSegmenter` (4-phase; healthcheck-abort;
`_probe_fps_width` fallback) |
tests\test_motion_segmenter.py:32:_CAP_PROP_FPS = "CAP_PROP_FPS"
tests\test_motion_segmenter.py:52:    cv2_mock.CAP_PROP_FPS = _CAP_PROP_FPS
tests\test_motion_segmenter.py:63:        if prop == _CAP_PROP_FPS:
tests\test_person_detector.py:19:        if prop == cv2.CAP_PROP_FPS:
tests\test_person_detector.py:74:        if prop == cv2.CAP_PROP_FPS:
tests\test_person_detector.py:105:    cap.get.side_effect = lambda p: {cv2.CAP_PROP_FPS: 30.0,
cv2.CAP_PROP_FRAME_WIDTH: 1000.0,
backend\reporting\harvest.py:50:        fps_str = s.get("r_frame_rate", "0/1")
backend\reporting\harvest.py:212:            vm["fps"] = 30.0  # matches the runners' fallback
(_probe_fps_width -> 30.0 / 1280)
backend\reporting\spec.yaml:87:      run (it probes the file) or ffprobe couldn't read
r_frame_rate. Confirm the real
tests\test_reporting.py:82:                {"codec_type": "video", "width": 1920, "height":
1080, "r_frame_rate": "30/1"},
tests\test_reporting.py:91:                {"codec_type": "video", "width": 1280, "height": 720,
"r_frame_rate": "30/1"},
tests\test_reporting_harvest.py:37:    "streams": [{"width": 1920, "height": 1080,
"r_frame_rate": "60000/1001", "codec_name": "hevc"}],
backend\pipeline\segmenters\motion.py:49:        fps = cap.get(cv2.CAP_PROP_FPS)
backend\pipeline\validators\resolution_fps.py:27:        fps_str = stream.get("r_frame_rate",
```

```
"0/1")
backend\pipeline\segmenters\trajectory_hybrid.py:89:    def _probe_fps_width(video_path: str) ->
tuple:
backend\pipeline\segmenters\trajectory_hybrid.py:92:        fps = cap.get(cv2.CAP_PROP_FPS) if
cap.isOpened() else 0.0
backend\pipeline\segmenters\trajectory_hybrid.py:187:        fps, frame_width =
self._probe_fps_width(video_path)
backend\pipeline\validators\scene_cut.py:23:        fps = cap.get(cv2.CAP_PROP_FPS)
tests\test_tracknet_hybrid.py:27:    fn = patch.object(TrackNetHybridSegmenter,
"_probe_fps_width", return_value=(30.0, 1280.0))(fn)
tests\test_tracknet_hybrid.py:157:def test_probe_fps_width_fallbacks():
tests\test_tracknet_hybrid.py:163:        fps, width =
TrackNetHybridSegmenter._probe_fps_width("missing.mp4")
tests\test_trajectory_hybrid_equivalence.py:49:         patch.object(cls, "_probe_fps_width",
return_value=(30.0, 1280.0)), \
backend\pipeline\detectors\person_detector.py:160:        fps = cap.get(cv2.CAP_PROP_FPS)
backend\pipeline\detectors\person_detector.py:281:        fps = cap.get(cv2.CAP_PROP_FPS)
backend\pipeline\detectors\person_detector.py:324:        fps = cap.get(cv2.CAP_PROP_FPS) if
cap.isOpened() else 0.0
tests\test_yolo_hybrid.py:20:        if prop == cv2.CAP_PROP_FPS:
```

17. assistant (2026-06-20T09:01:44.359Z)

Very important finding from this grep. There IS a robust ffprobe-based fps reader in the codebase: `backend/pipeline/validators/resolution_fps.py:27` reads `r_frame_rate`, and the report's `fps_fallback_used` flag (CODE_MAP.md:290) and `backend/reporting/harvest.py` (`fps_str = s.get("r_frame_rate")`). So the proposed fix (an ffprobe-based fps probe) is feasible with existing helpers.

Crucially, CODE_MAP.md already documents this exact hazard at line 290:
> **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`. The report's `fps_fallback_used` flag surfaces this.

This is a known, documented foot-gun. The finding correctly identifies that in the NEW worker training path (`__main__.py:252`), the fallback is used WITHOUT any `fps_fallback_used` surfacing ? it gets written verbatim into the training manifest.

Let me verify the resolution_fps validator to confirm the robust ffprobe path exists and the comparison to `get_video_duration` is fair.

18. user (2026-06-20T09:01:45.782Z)

```
1       from backend.pipeline.validators.base import VideoValidator, ValidationResult
2       from typing import Dict, Any
3
4       class ResolutionFPSValidator(VideoValidator):
5           @property
6           def name(self) -> str:
7               return "resolution_fps_check"
8
9           @property
10          def description(self) -> str:
11              return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14              meta = context.get("ffprobe_meta")
15              if not meta or not meta.get("streams"):
16                  return ValidationResult(
17                      validator_name=self.name,
18                      passed=False,
19                      message="Video metadata is missing. Run format_check validator first."
20                  )
```

```
21
22              stream = meta["streams"][0]
23              width = int(stream.get("width", 0))
24              height = int(stream.get("height", 0))
25
26              # Parse frame rate (e.g. "30/1" or "30000/1001")
27              fps_str = stream.get("r_frame_rate", "0/1")
28              try:
29                  num, den = map(int, fps_str.split("/"))
30                  fps = num / den if den != 0 else 0
31              except ValueError:
32                  fps = 0.0
33
34              details = {
35                  "width": width,
36                  "height": height,
37                  "fps": round(fps, 2)
38              }
39
40              # Check bounds from config
41              val_cfg = config.get("validation", {})
42              min_w = val_cfg.get("min_resolution_width", 1280)
43              min_h = val_cfg.get("min_resolution_height", 720)
44              min_fps = val_cfg.get("min_fps", 29.9)
45
46              if width < min_w or height < min_h:
47                  return ValidationResult(
48                      validator_name=self.name,
49                      passed=False,
50                      message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
51                      details=details
52                  )
53
54              if fps < min_fps:
55                  return ValidationResult(
56                      validator_name=self.name,
57                      passed=False,
58                      message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
59                      details=details
60                  )
61
62              return ValidationResult(
63                  validator_name=self.name,
64                  passed=True,
65                  message="Video resolution and frame rate meet minimum standards.",
66                  details=details
67              )
68
```

19. user (2026-06-20T09:01:46.439Z)

```
155
156
157     def test_probe_fps_width_fallbacks():
158         with patch("backend.pipeline.segmenters.trajectory_hybrid.cv2.VideoCapture") as
mock_cap:
159             bad = MagicMock()
160             bad.isOpened.return_value = False
161             bad.get.return_value = 0.0
162             mock_cap.return_value = bad
163             fps, width = TrackNetHybridSegmenter._probe_fps_width("missing.mp4")
164             assert fps == 30.0 and width == 1280.0
165
```

```
166
167     @_patched
168     def test_skip_ai_handoff_emits_candidates_as_rallies(mock_runner_cls, mock_probe,
mock_dur,
169                                                          mock_extract,
mock_provider_registry, mock_env):
```

20. assistant (2026-06-20T09:02:05.448Z)

I now have full confirmation. The existing test `test_probe_fps_width_fallbacks` (lines 157-164) explicitly documents the silent fallback behavior: when cv2 can't open the file, `_probe_fps_width` returns `(30.0, 1280.0)` with no error or signal.

Let me assess the severity question carefully. The finding is rated "major". I need to weigh:

1. **Real failure scenario confirmed:** cv2 fails to open / reports 0 fps (unusual codec, container cv2 can't read but ffmpeg/WASB can, or any open failure) -> silent 30.0/1280.0 -> written verbatim into the training manifest -> `build_frame_arrays`/`features` use fps for frame->time mapping AND speed normalization -> corrupted label alignment with no error surfaced. This is a genuine data-corruption-without-error path.

2. **The asymmetry the finding flags is accurate:** `get_video_duration` (3 lines earlier, line 249... actually `compute_video_id` at 249, `get_video_duration` is used in segmenter's `process_video`) uses robust ffprobe; `_probe_fps_width` uses cv2 with a silent default. And a robust ffprobe fps reader already exists in the repo (`resolution_fps.py`).

3. **Severity considerations (against over-rating):**
   - This is a known, documented hazard (CODE_MAP.md:290) ? pre-existing in `_probe_fps_width`, not newly introduced. But the NEW code wires it into a NEW consumer (training manifest) where the consequence (corrupt training/eval data, no error) is more severe than the inference path (where the report surfaces `fps_fallback_used`).
   - The detector path is gated behind `--trajectory-source detector` (default is `synth`), so the CPU e2e / default path is unaffected. It only bites on the real GPU-box training run.
   - WASB itself extracts frames (likely via ffmpeg/cv2 too) ? if the codec is truly unreadable, WASB may also fail and produce no trajectory (the `if not traj: skip` guard at line 254 would catch a total failure). The dangerous case is the narrower one: cv2 mis-reports fps (e.g. returns 0 or a wrong value) while ffmpeg/WASB still decode correctly. That's a real but narrower window than "any open failure".

The finding is a real, correctly-located issue with an accurate failure mechanism. The 30fps-vs-60fps mis-alignment is concrete: `times = idx / fps` and `labels_for` compares against golden interval seconds, so a 2x fps error doubles every trajectory time relative to labels. On a multi-fps corpus (the codebase explicitly mixes 30 and 59.94 fps per the rally_seq.py docstring), feeding a guessed 30 into a 59.94fps video corrupts that video's entire training contribution silently.

This is a legitimate "major"-tier finding: it silently corrupts training/eval data on the real detector path with no error surfaced, the fix is straightforward, and the robust probe already exists in-repo. The only thing slightly favoring "minor" is that it's an opt-in, owner-run-on-GPU-box path ? but the consequence (silent training-data corruption that looks like a normal run) is exactly the class of bug that warrants major.

21. user (2026-06-20T09:02:20.986Z)

Structured output provided successfully

22. assistant (2026-06-20T09:02:28.781Z)

I've verified the finding against the actual code and confirmed it is real. The structured output has been submitted with verdict `real` and severity `major` (unchanged).

Key files and lines that ground the verdict:
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py:252` ? the call site that writes the probed fps/frame_width verbatim into the training manifest

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py:88-95` ? `_probe_fps_width` silently falling back to (30.0, 1280.0)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\features\rally_seq.py:115` ? `times = idx / fps`, the frame->seconds mapping that mis-aligns labels when fps is wrong
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seq_proto.py:53-60,113-114` ? `labels_for(times, gts)` comparing fps-derived times against golden labels
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\resolution_fps.py:27` ? an existing robust ffprobe `r_frame_rate` reader the proposed fix can reuse

The finding is accurate in location, mechanism, and the get_video_duration asymmetry it cites. I kept it at major rather than escalating because it is opt-in behind `--trajectory-source detector` (default `synth` is unaffected), but the consequence ? silent training-data corruption that looks like a normal run ? warrants the major rating.